CMS 375 – Database Design and Development

Project Final Report

Spring 2026

All-In-One Movies & TV Platform

Developed by **MTM Studios**

Team Members:

Travis Fine

Manal Marhit

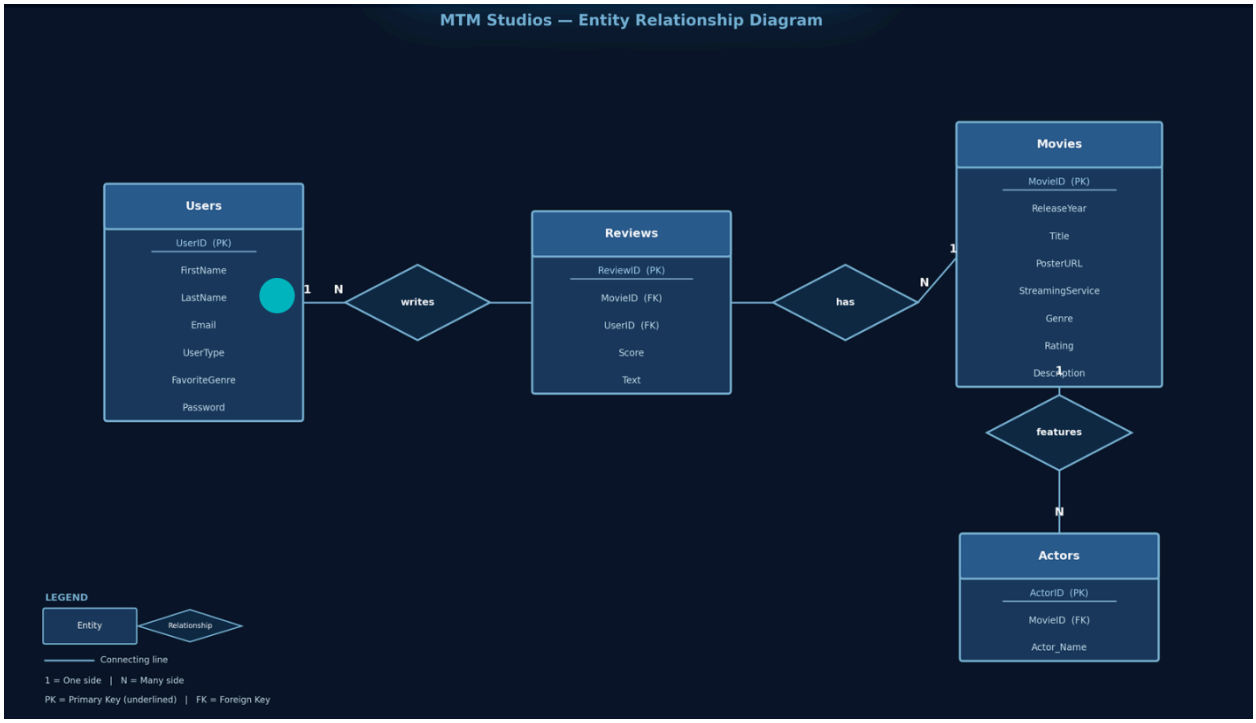
Mackenzie Yates

1. <u>Introduction:</u>

We have all been there. You want to watch a movie but there's just too many options, sites, and conflicting reviews. We at MTM Studios want to solve this problem forever by creating a site in which everything can be searched through with ease. Our site aims to take the hassle out of movie browsing, allowing all platforms and genres to exist in the same space for our users. In this report, we will go through the structure and implementation of our easy-browse movie site in detail.

2. <u>ER Diagram:</u>

The Entity-Relationship (ER) diagram for MTM Studios illustrates the structure and relationships between the four database entities: Users, Movies, Reviews, and Actors:



Entities and Attributes:

Users stores all registered user information, with UserID serving as the primary key. Movies stores all film data, with MovieID as the primary key. Reviews serves as the junction entity between Users and Movies, with ReviewID as the primary key and both MovieID and UserID as foreign keys. Actors is a dependent entity of Movies, with ActorID as the primary key and MovieID as a foreign key.

Relationships:

A User *writes* many Reviews, forming a one-to-many relationship enforced by the UserID foreign key in the Reviews table. A Movie *has* many Reviews, also one-to-many, enforced by the MovieID foreign key in the Reviews table. A Movie *features* many Actors, one-to-many, enforced by the MovieID foreign key in the Actors table.

Design Decisions:

The Actors table was separated from Movies to satisfy Third Normal Form. Originally, actor names were stored as a comma-separated string in a single column inside the Movies table, which violated First Normal Form by placing multiple values in one field. Moving actors into their own table eliminates that redundancy and makes each actor individually queryable. The Reviews table contains no redundant data, as all user and movie information is accessed through foreign keys rather than being duplicated across tables.

3. <u>Database Design</u>

Our database consisted of four different tables: the users table, the movies table, the reviews table, and the actors table. Each table has its own separate values, with the users table consisting of Users(UserID, FirstName, Lastname, email, usertype, favoritegenre, password), with the primary key being UserID. The movies table consists of movies(MovieID, Releaseyear, title, poster url, streamingservice, genre, rating, description), and the primary key is MovieID. The Reviews table consists of reviews(ReviewID, MovieID, UserID, Score, Text), the primary key for this one being ReviewID and the foreign keys being MovieID and UserID. And finally, the Actor table consists of Actors(ActorID, MovieID, Actor_Name), with the primary key for this table being ActorID and the foreign key being MovieID. Each of these tables has around 50 entries, with most of the data being pulled from IMDB and put mainly into the Movie and actor tables.

4. <u>Implementation Details with codes</u>

The backend of MTM Studios is built entirely in PHP using the MySQLi extension. Every page that interacts with the database follows the same secure pattern: establish a connection through db_connect.php, prepare a parameterized query, bind user input as parameters, execute the query, and retrieve the results. This consistent approach ensures that no user-supplied input is ever directly interpolated into a SQL query anywhere in the application, which eliminates the risk of SQL injection across the entire codebase.

Authentication and Session Management

Authentication is handled across two files: index.php and login.php. Both implement the same secure login flow. When a user submits their credentials, the backend uses a prepared statement to retrieve the user record rather than constructing the query with raw input:

```php
$stmt = $conn->prepare("SELECT * FROM Users WHERE UserID = ?");
$stmt->bind_param("s", $user_id);
$stmt->execute();
$result = $stmt->get_result();
```

The ? placeholder means the user's input is treated as pure data by the database engine and can never be interpreted as part of the SQL command itself. Once the record is retrieved, the submitted password is verified against the stored bcrypt hash using PHP's built-in password_verify() function:

```php
if ($result->num_rows == 1) {
    $user = $result->fetch_assoc();
    if (password_verify($password, $user['Password'])) {
        $_SESSION['logged_in']  = true;
        $_SESSION['user_id']    = $user['UserID'];
        $_SESSION['user_type']  = $user['UserType'];
        $_SESSION['fav_genre']  = $user['FavoriteGenre'];
        header("Location: index.php");
        exit();
    } else {
        $error = "Incorrect password.";
    }
} else {
    $error = "User not found.";
}
$stmt->close();
```

The actual password is never stored in the database, only its bcrypt hash. Even if the database were compromised, an attacker would see a string beginning with $2y$10$ which cannot be reversed to recover the original password. On successful login, the user's ID, type, and favorite genre are written to the PHP session so that every subsequent page load can identify who is logged in and what permissions they have without querying the database again.

The application also supports guest access. A guest session is created with a Guest flag that prevents access to write operations such as submitting reviews. Every protected page checks for a valid session at the top of the file and redirects unauthenticated visitors to the login page.

User Registration:

New account creation is handled by register.php. The form collects first name, last name, email address, username, favorite genre, and password with confirmation. Before inserting anything into the database, the backend performs multiple layers of server-side validation:

It checks that all fields are present, validates email format using PHP's FILTER_VALIDATE_EMAIL, enforces a username length between 3 and 20 characters, ensures the username contains only letters, numbers, and underscores, and confirms the two password

fields match. It then checks for duplicate usernames and duplicate email addresses using separate prepared statements:

```
$stmt = $conn->prepare("SELECT UserID FROM Users WHERE UserID = ?");
$stmt->bind_param("s", $username);
$stmt->execute();
$stmt->store_result();
if ($stmt->num_rows > 0) {
    $error = "That username is already taken. Please choose another.";
}
```

If all validations pass, the password is hashed before insertion:

```
// Insert new user with hashed password
$hashed = password_hash($password, PASSWORD_DEFAULT);
$stmt = $conn->prepare("INSERT INTO Users (UserID, FirstName, LastName, Email,
        UserType, FavoriteGenre, Password) VALUES (?, ?, ?, ?, 'Regular', ?, ?)");
$stmt->bind_param("ssssss", $username, $first_name, $last_name, $email, $fav_genre, $hashed);
```

After a successful insert the user is automatically logged in and redirected to the home page, so they do not have to go through a separate login step after registering.

Movie Browsing and Dynamic Filtering

The movies.php file is the main browsing interface. In its default state it displays all movies grouped by genre in horizontal scrollable rows. When the user applies filters, the view switches to a flat grid. The filtering system supports four independent parameters: a keyword that searches across movie titles, descriptions, and actor names; a genre exact match; a release year integer filter; and a streaming service partial match.

These filters are applied through a dynamically constructed prepared statement. Each active filter appends a condition to the query and adds its value to a parameters array. The full query is then prepared and executed in a single call:

```
if ($search !== '') {
    $like = "%$search%";
    $sql .= " AND (Title LIKE ? OR Description LIKE ? OR MovieID IN (SELECT MovieID FROM Actors WHERE Actor_Name LIKE ?))";
    $params[] = $like;
    $params[] = $like;
    $params[] = $like;
    $types .= 'sss';
```

```
$stmt = $conn->prepare($sql);
if (!empty($params)) {
    $stmt->bind_param($types, ...$params);
}
```

The sort order is validated against a whitelist of allowed values before being appended to the query, ensuring that even the ORDER BY clause cannot be manipulated by user input. This design means a single secure query handles all possible filter combinations simultaneously.

Review Submission System

The review system operates through two separate files depending on where the user submits from. Reviews submitted from an individual movie's page are handled by movie.php, while reviews submitted through the homepage modal are handled by submit_review.php. Both follow the same logic.

Before inserting a review, the system checks whether the current user has already reviewed that movie using a prepared statement:

```php
$stmt = $conn->prepare("SELECT ReviewID FROM Reviews WHERE UserID = ? AND MovieID = ?");
$stmt->bind_param("ss", $user_id, $movie_id);
$stmt->execute();
$stmt->store_result();
if ($stmt->num_rows > 0) {
    echo json_encode(['success' => false, 'message' => 'You have already reviewed this movie.']);
    $stmt->close();
    exit();
}
```

If no duplicate exists, a unique review ID is generated by querying the current maximum and incrementing it. The review is then inserted with a fully parameterized query:

```php
$stmt = $conn->prepare("INSERT INTO Reviews (ReviewID, MovieID, UserID, Score, Text) VALUES (?, ?, ?, ?, ?)");
$stmt->bind_param("sssis", $rev_id, $movie_id, $user_id, $score, $text);
$stmt->execute();
$stmt->close();
```

The submit_review.php endpoint returns a JSON response so the homepage modal can display a success or error message without reloading the page, keeping the user experience seamless.

Live Movie Search

The search_movies.php file is a lightweight JSON endpoint that powers the live search inside the homepage rate-it modal. When a user types a movie title in the modal's search field, JavaScript calls this endpoint in the background without any page reload. The endpoint wraps the search term in wildcard characters and executes a prepared LIKE query:

```php
$search_param = "%$q%";
$stmt = $conn->prepare("SELECT MovieID, Title, Genre, Rating, ReleaseYear FROM Movies WHERE Title LIKE ? ORDER BY Rating DESC LIMIT 10");
$stmt->bind_param("s", $search_param);
$stmt->execute();
$result = $stmt->get_result();
```

Results are returned as a JSON array and rendered instantly in the modal as clickable cards. The user selects a movie, writes their review, and submits, all without leaving the home page. This is a clear example of the frontend and backend working together in real time, with PHP handling the data retrieval and security while JavaScript handles the user interface updates.

Movie Poster System:

The poster system was developed by a teammate and spans three files: poster_helper.php, fetch_poster.php, and cache_poster.php. Its core design principle is that the page should never block while waiting for an external API response.

poster_helper.php exposes a renderPosterImg() function that checks whether a valid poster URL is already cached in the database. If a real URL exists it renders an image tag directly. If not, it renders a styled fallback card containing the genre and title, and attaches a data-fetch attribute containing the movie's IMDb ID. JavaScript on each page reads this attribute after the page has loaded and calls fetch_poster.php asynchronously.

fetch_poster.php is the server-side OMDB API proxy. It first checks the database cache again to avoid redundant API calls. If no cached URL exists it calls the OMDB API using cURL, with fallback to file_get_contents, parses the response, saves the poster URL back to the database, and returns it as JSON.

cache_poster.php handles direct writes from the client-side JavaScript path. After JavaScript fetches a poster URL directly from OMDB using the API key embedded in the page, it silently calls this endpoint to save the URL to the database. Both the IMDb ID and the URL are validated with regular expressions before any database write occurs.

Together these three files ensure that each movie's poster is fetched from OMDB exactly once and then served from the local database on every subsequent page load.

Password Hashing Utility

hash_passwords.php was a one-time utility written to address the plain text passwords present in the original seeded dataset. It iterated through every user in the database, checked whether the stored password already had the $2y$ bcrypt prefix, and if not hashed it using password_hash() and updated the record. This file was only ever intended to be run once and includes a warning to delete it afterward. All passwords registered through the application after this point are hashed at the point of insertion in register.php, so this utility has no ongoing role in the application.

5. <u>Challenges</u>

When it comes to the challenges we faced while working on this project, a few initial things come to mind. One of the first things we had to work through was our group schedule and time management. All three of our members have very different schedules, making meetings and discussion something that required a bit more creativity than usually desired. Our group chat ended up getting much usage, with the majority of our communication happening inside. We also used Google Meets in order to discuss and practice our presentation slides. These online meetings allowed us to work as a team even when time was against us. Another challenge we faced was the implementation of specific features. The first of these features was the displaying of a movie’s poster on our site. Images make a huge difference in the potential

allure of a movie, with the poster being the most prevalent of these images. In order to implement this, we had to work with an external API that pulls the movie posters from IMDB using the IMDB movie ID which we already used in our database to pull review data. Another feature that was not particularly simple to implement was our security system. As of now, our site uses hashcode encryption in order to keep passwords secure for all users. This feature (because we did not initially have it) took time to fit into our website. The last major challenge we ran into during our project was the normalization of our data. In our initial drafts, we had three tables; movies, users, and reviews. We realized during our normalization process that having the actors be listed inside of the movies table violated the normalization rules and needed to be adjusted. We ended up using a fourth table, actors, in order to store the actor data separate from the movies table in order to meet the normalization requirements. Besides these challenges, our site development went extremely smoothly and our team is most certainly effective in our tasks.

6. Future Work

For our future work, most of what we want to implement are things that will amke out website more user-friendly. In order to do this, we will be adding more movies and genres to our database so that our users have a wider selection to choose from. We also want to add a chatbot function that will allow users to get specific movie recommendations based on their movie preferences. Going along with the theme of making our website more personalised, we will be adding a function that allows users to fully customise their profile, and we will also be implementing a system that allows users to recover or reset their passwords should they lose or forget them.